

Teoremas, Proposiciones y Corolarios

Bander

Julio 2025

Este es un resumen de los teoremas en mis palabras y con sobreexplicación seguramente para así yo lo entiendo. No pretendo reemplazar la documentación oficial, si no que es para tener una explicación para mi yo del futuro.

Teoremas importantes

- Teorema de Cook Levin
- Jerarquía de Tiempos Determinísticos
- Jerarquía de Tiempos no Determinísticos
- Teorema de Ladner
- Jerarquía de Espacio
- Teorema de Savitch
- Teorema de Immerman-Szelepcsényi
- Teorema de Baker, Gill, Solovay
- Teorema de Karp-Lipton

Proposición 1: Sea $f : \{0, 1\}^* \rightarrow \{0, 1\}^*$, sea T una función construible en tiempo y sea Γ un alfabeto. Si f es computable en tiempo $T(n)$ por una máquina $M = (\Gamma, Q, \delta)$, entonces f es computable en tiempo $O(\log|\Gamma| \cdot T(n))$ por una máquina $M' = (\Sigma, Q', \delta')$ donde $\Sigma = \{0, 1, \triangleright, \square\}$ es el alfabeto estándar.

Demostración:

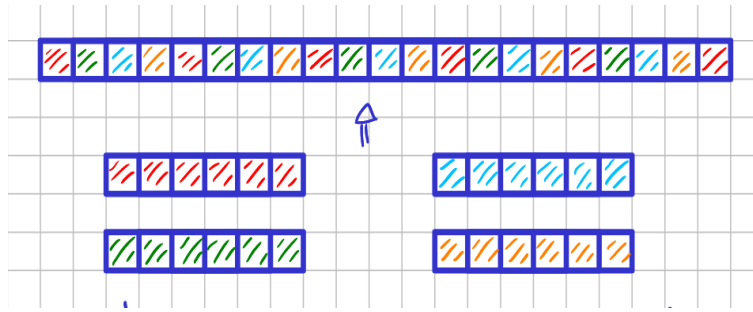
$|\Gamma|$ es la cantidad de estados que hay originalmente, para codificar cada estado de Q en Q' se usa el alfabeto Σ (binario), por lo cual conlleva $\log_2|\Gamma|$ bits por estado (por convencion usamos que $\log_2|\Gamma| = \log|\Gamma|$ bits).¹ Eso se traslada a δ' también, por lo cual, lo que antes llevaba un símbolo perteneciente a Γ ahora lleva $\log_2|\Gamma|$ símbolos de Σ (pej.: $A = 1010$).

Proposición 2: Sea $f : \{0, 1\}^* \rightarrow \{0, 1\}^*$ y sea T una función construible en tiempo. Si f es computable en tiempo $T(n)$ por una máquina estándar de $k \geq 3$ cintas (entrada, salida y $k - 2$ cintas de trabajo), entonces f es computable en tiempo $O(T(n)^2)$ por una máquina de cinta única.

Demostración:

Puedo alternar los símbolos de las k cintas en la cinta única y usar símbolos característicos para indicar dónde está la cabeza de cada cinta.

En la posición i está el caracter $\lceil \frac{i}{k} \rceil$ de la cinta $i \bmod k$.



Queda en $O(T(n)^2)$ debido a que por cada paso de δ debo primero ubicar cada cabeza para ver que accionar y después modificar cada cabeza como lo indique δ . O sea, por cada paso recorro toda la cinta 2 veces.

Proposición 3: Sea $f : \{0, 1\}^* \rightarrow \{0, 1\}^*$ y sea T una función construible en tiempo. Si f es computable en tiempo $T(n)$ por una máquina estándar, entonces hay una máquina oblivious que computa f en tiempo $O(T(n)^2)$.

Demostración:

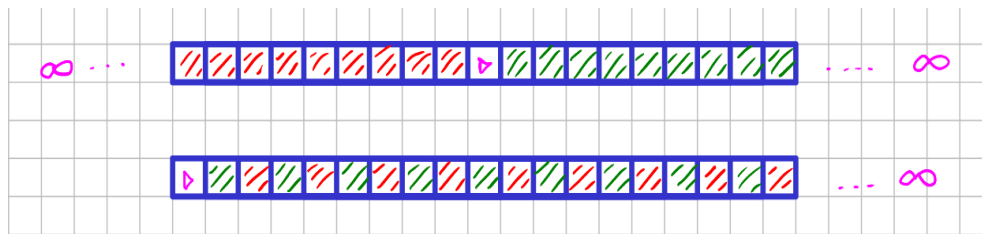
La máquina oblivious mueve un patrón fijo en cada paso (barre toda la cinta) y cambia la cinta según los cambios que se piden. Es muy similar a la [proposición 2](#).

Proposición 4:

Sea $f : \{0, 1\}^* \rightarrow \{0, 1\}^*$ y T una función construible en tiempo. Si f es computable por una máquina con cintas bi-infinitas en tiempo $T(n)$, entonces f es computable por una máquina estándar en tiempo $O(T(n))$.

Demostración:

Se puede doblar la cinta bi-infinita de manera que rebota en el símbolo $\triangleright$ para ver ambos infinitos.



¹La base no importa en la notación big O para los logaritmos ya que difieren entre si por una constante multiplicativa debido a cómo se puede cambiar la base de un logaritmo ($\log_b n = \frac{\log_k n}{\log_k b}$)

Teorema 1: [Turing 1936] *halt no es computable.*

Recordatorio:

$$\text{halt}(x) = \begin{cases} 1 & \text{si la máquina con entrada } x \text{ termina } (M_x(x)) \\ 0 & \text{si no} \end{cases}$$

Demostración:

Sale por diagonalización. Tengo que:

	M_1	M_2	M_3	$\dots$
1	$M_1(1)$			
2		$M_2(2)$		
3			$M_3(3)$	
$\vdots$				$\ddots$

Defino entonces M tal que $M(x)$ termina sii $\text{halt}(x) = 0$.

$M(\langle M \rangle)$ termina $\iff \text{halt}(\langle M \rangle) = 0 \iff M(\langle M \rangle)$ no termina. **Absurdo!**

Teorema 2: *Existe una máquina U que computa la función $u(\langle i, x \rangle) = M_i(x)$. Más aún, si M_i con entrada x termina en t pasos, entonces U con entrada $\langle i, x \rangle$ termina en $c \cdot t \cdot \log(t)$ pasos, donde c depende solo de i .*

Demostración:

Pone en cada cinta de trabajo la simulación de la máquina estándar de M . O sea, en

- #1: La entrada x .
- #2: Cinta de trabajo de M .
- #3: Estado de M .

Si #3 = $[q_f]$ termina la ejecución M .

Por cada paso de M busco δ en la entrada de la máquina U .

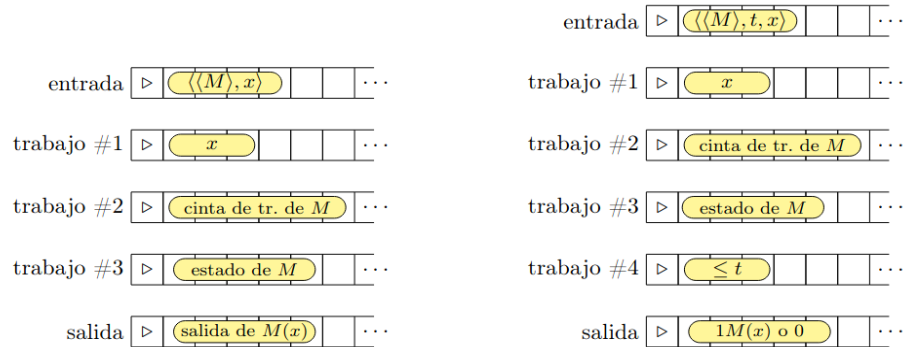


Figura 12: Izquierda: la simulación que hace U (con 3 cintas de trabajo) de M (con una única cinta de trabajo) y entrada x . Derecha: la simulación que hace $\tilde{U}$ (con 4 cintas de trabajo) de M (con una única cinta de trabajo), entrada x hasta el tiempo t .

Teorema 3: *Existe una máquina $\tilde{U}$ que computa la función $\tilde{u}(\langle i, t, x \rangle)$ en tiempo $c \cdot t \cdot \log(t)$, donde c depende solo de i .*

Demostración:

Es muy similar al **teorema 2** pero con una cinta más de trabajo para llevar registro de i (cantidad de pasos en la simulación).

Teorema 4: $P \subseteq NP$ **Demostración:**

Sea $\mathcal{L} \in P$. $\mathcal{L}(M)$, con M una máq. det. que corre en tiempo polinomial.

Tomás el polinomio p de la definición como $p(|x|) = 0$. Defino M' det. y que corre en tiempo poly. y el certificado $c = \varepsilon$ (donde ε es la cadena vacía), entonces tengo que M' con entrada $\langle x, c \rangle$ copia el comportamiento de M con entrada x .

$$\begin{aligned} x \in \mathcal{L} &\iff M(x) = 1 \\ &\iff M'(\langle x, c \rangle) = 1 \\ &\iff \exists c. c \in \{0, 1\}^0 \text{ tal que } M'(\langle x, c \rangle) \text{ (Definición de NP)} \end{aligned}$$

Teorema 5: $NP = \bigcup_{c \in \mathbb{N}} \text{NDTime}(n^c)$ **Demostración:** $\bigcup_{c \in \mathbb{N}} \text{NDTime}(n^c) \subseteq NP$:

Sea $\mathcal{L} \in \bigcup_{c \in \mathbb{N}} \text{NDTime}(n^c)$ quiero ver que $\mathcal{L} \in NP$. Sea N una máq. no-det. y p un polinomio tal que N corre en tiempo $p(n)$ y $\mathcal{L}(N)$.

Existe una máq. det. M tal que M con entrada $\langle x, u \rangle$ verifica que u sea la codificación del cómputo aceptador de N a partir de x . M simula N con entrada x paso a paso, y en cada iteración i usa la primera o la segunda componente de δ según el valor de $u(i)$ para saber qué hacer.

Luego, $x \in \mathcal{L} \iff \exists u. u \in \{0, 1\}^{p(|x|)}$ tal que $M(\langle x, u \rangle) = 1$. Como M corre en tiempo polinomial (ya que es solo recorrer las decisiones en N con u), concluimos que $\mathcal{L} \in NP$.

 $NP \subseteq \bigcup_{c \in \mathbb{N}} \text{NDTime}(n^c)$:

Sea $\mathcal{L} \in NP$ quiero ver que $\mathcal{L} \in \bigcup_{c \in \mathbb{N}} \text{NDTime}(n^c)$. Para esto uso la máq. M det. que corre en tiempo poly de la definición y la simulo con una no-det. que hace lo mismo pero inventando el certificado u por lo que te queda que corre en tiempo poly al ser que M corría en tiempo poly.

Teorema 6: Existe una máquina no-determinística NU tal que NU acepta $\langle i, x \rangle$ sii N_i acepta x y si N_i corre en tiempo $T(n)$ entonces $NU(\langle i, x \rangle)$ decide si N_i acepta o rechaza x en $c \cdot T(|x|)$ pasos, donde c depende solo de i .

Demostración:

Similar al teorema 2. No tiene el logaritmo porque puede adivinar no deterministamente la secuencia de elecciones que seguiría N_i para aceptar x .

Teorema 7: La relación $\leq_p$ es transitiva.

Demostración:

Sea $\mathcal{L} \leq_p \mathcal{L}'$ vía g quiero ver que $\mathcal{L} \leq_p \mathcal{L}''$.

$$x \in \mathcal{L} \iff f(x) \in \mathcal{L}' \iff g(f(x)) \in \mathcal{L}''$$

Ahora quiero ver que $g \circ f$ es computable en tiempo polinomial respecto $|x|$.

Sea M_f, M_g tal que M_f computa f en tiempo poly y M_g computa g en tiempo poly.

$M_{g \circ f} : \langle x \rangle$
 $y := M_f(x) \quad O(n^c)$ A lo sumo su salida es polinomial respecto n ($n = |x|$)
 return $M_g(y) \quad O((n^c)^d) = O(n^{cd})$ A lo sumo es poly respecto a $|y|$

Teorema 8: Si $NP\text{-hard} \cap P \neq \emptyset$, entonces $P = NP$.

Demostración:

Si $\mathcal{L} \in NP\text{-hard} \cap P$, entonces puedo tomar cualquier $\mathcal{L}' \in NP$ y reducirlo a $\mathcal{L}$, por lo cuál tengo una f computable poly tal que:

$$x \in \mathcal{L}' \iff f(x) \in \mathcal{L}$$

Entonces a la $\mathcal{L}'(M_{\mathcal{L}'})$ la declaro como:

$M_{\mathcal{L}'} : \langle x \rangle$
 $y := M_f(x) \quad O(n^c)$
 $M_{\mathcal{L}}(y) \quad O(n^{cd})$

$M_{\mathcal{L}'}$ es entonces una máq. det. que corre en tiempo poly por lo que $\mathcal{L}' \in P$. O sea $P = NP$, al ser que $\mathcal{L}'$ es genérico.

Teorema 9: Si $\mathcal{L} \in \text{NP-completo}$, entonces $\mathcal{L} \in \text{P} \iff \text{P} = \text{NP}$.

Demostración:

$\mathcal{L} \in \text{P} \Rightarrow \text{P} = \text{NP}$:

Si $\mathcal{L}$ es NP-completo y a su vez $\mathcal{L} \in \text{P}$, lo único que agrega que $\mathcal{L} \in \text{NP-completo}$ es que $\mathcal{L} \in \text{NP-hard}$.² Luego, por el **teorema 8** obtengo que $\text{P} = \text{NP}$.

$\text{P} = \text{NP} \Rightarrow \mathcal{L} \in \text{P}$:

Si $\text{P} = \text{NP}$, entonces todos los problemas de NP se pueden resolver en tiempo poly, en particular también todos los NP-completos.

Proposición 6: TMSAT $\in$ NP-completo.

Recordatorio:

$\text{TMSAT} = \{ \langle y, x, 1^n, t \rangle \mid \exists u \in \{0, 1\}^n \ M_y(xu) = 1 \text{ en a lo sumo } t \text{ pasos} \}$

Demostración:

Sea $\mathcal{L} \in \text{NP}$:

$$x \in \mathcal{L} \iff \exists u \in \{0, 1\}^{p(|x|)} : M(x, u) = 1 \quad \text{definición de } \mathcal{L} \text{ en NP}$$

$$x \in \mathcal{L} \iff f(x) \in \text{TMSAT} \quad \text{definición } \mathcal{L} \leq_p \text{TMSAT}$$

La f de la reducción sería:

$$f(x) = \langle \langle M \rangle, x, 1^{p(|x|)}, 1^{q(|x|+p(|x|))} \rangle \quad (\text{lo cual es computable en tiempo poly})$$

$$f(x) \in \text{TMSAT} \iff \langle \langle M \rangle, x, 1^{p(|x|)}, 1^{q(|x|+p(|x|))} \rangle \in \text{TMSAT}$$

$$\iff \exists u \in \{0, 1\}^{p(|x|)} : M(x, u) = 1 \text{ en } \leq q(|x|, p(|x|)) \text{ pasos}$$

$$\iff x \in \mathcal{L}$$

Proposición 7: Sea $\varphi(p_1, \dots, p_n)$ una fórmula booleana. Si v y v' son dos valuaciones tales que $v(p_i) = v'(p_i)$ para $i \in \{1, \dots, n\}$, entonces $v \models \varphi(p_1, \dots, p_n)$ sii $v' \models \varphi(p_1, \dots, p_n)$.

Demostración:

$v(p_i) = v'(p_i)$ significa que la valuación evalúa a las variables proposicionales con el mismo valor. La proposición sale inmediata.

Teorema 10: SAT, 3SAT $\in$ NP

Demostración:

$M : \langle x, u \rangle$

x representa una fórmula booleana

if $(x = \langle y \rangle)$ y φ tiene m variables proposicionales)

ret $u \upharpoonright m \models \varphi$ ($\upharpoonright$ toma los primeros m elementos de u)

Proposición 8: Para toda $F : \{0, 1\}^\ell \rightarrow \{0, 1\}$ existe una fórmula booleana $\varphi_F(p_1, \dots, p_\ell)$ en CNF tal que $v \models \varphi$ sii $F(v) = 1$ para todo $v \in \{0, 1\}^\ell$. Más aún, φ_F se computa en tiempo polinomial a partir de $\langle F \rangle$ y tiene tamaño $O(\ell 2^\ell)$.

Demostración:

Concateno todas las posibles combinaciones de ℓ bits ($\ell 2^\ell$) de manera que si $F(v)$ da 0, haya un $v \not\models (p_1 \wedge \neg p_2 \cdots \wedge p_n)$ y $(p_1 \wedge \cdots \wedge p_n)$ está en φ_F

Corolario 1: Para toda $F : \{0, 1\}^\ell \rightarrow \{0, 1\}^k$ existe una fórmula booleana $\varphi(p_1, \dots, p_{\ell+k})$ en CNF tal que $uv \models \varphi_F$ sii $F(u) = v$ para todo $u \in \{0, 1\}^\ell, v \in \{0, 1\}^k$. Más aún, φ_F se computa en tiempo polinomial a partir de $\langle F \rangle$ y tiene tamaño $O((\ell + k)2^{\ell+k})$.

Demostración:

Defino $G : \{0, 1\}^\ell \rightarrow \{0, 1\}^k$ tal que $G(u, v) = 1 \iff F(u) = v$ y aplico la **proposición 8**.

² $\mathcal{L}$ ya era NP, porque $\text{P} \subseteq \text{NP}$

Teorema 11: (Teorema de Cook Levin) $\text{SAT} \in \text{NP-hard}$.

Idea de la demostración: (no la explico toda porque es muy técnica)

$$\begin{aligned} x \in \mathcal{L} &\iff \exists u \in \{0,1\}^{p(|x|)} : M(x, u) = 1 \\ &\iff \text{Existe } u \in \{0,1\}^{p(|x|)} \text{ y existe un c\u00f3puto } C_0, \dots, C_{t(|xu|)} \text{ a partir de la entrada } xu \end{aligned}$$

Con el uso de mini-configuraciones $z_0, \dots, z_m \in \{0,1\}^k$ con k dependiente de la m\u00e1quina.

Con la entrada (xu) quiero ver si con f\u00f3rmulas de tama\u00f1o polinomial a la entrada puedo simular el c\u00f3puto de la m\u00e1q. M det. que decide a $\mathcal{L}$.

Las f\u00f3rmulas son:

1. La que representa la cinta de entrada. (xu)
2. La configuraci\u00f3n inicial de M a partir de (xu) . (q_0)
3. La evoluci\u00f3n en un paso de z_i a z_{i+1} . (δ)
4. La configuraci\u00f3n final de M aceptadora. $(q_f \text{ y el s\u00edmbolo en la cinta de trabajo es } 1)$

Corolario 2: $\text{SAT}, 3\text{SAT} \in \text{NP-completos}$.

Demostraci\u00f3n:

Sale del [teorema 10 y 11](#).

Proposici\u00f3n 9: $\text{INDSET} \in \text{NP-completo}$.

Demostraci\u00f3n:

Ver que $\text{INDSET} \in \text{NP}$ es medianamente f\u00e1cil, ahora ver que $\text{INDSET} \in \text{NP-hard}$ ten\u00e9s que hacerlo por definici\u00f3n ([nunca](#)) o por reducci\u00f3n polinomial de un NP-hard que ya sepamos.

Veamos que $3\text{SAT} \leq_p \text{INDSET}$:

$$\varphi = (\ell_{11} \vee \ell_{12} \vee \ell_{13}) \wedge \dots \wedge (\ell_{m1} \vee \ell_{m2} \vee \ell_{m3})$$

A lo sumo tengo $3m$ v\u00e9rtice (si todos los literales son distintos), uno por cada variable.

Defino una arista entre z y z' si est\u00e1n en la misma cl\u00e1usula de la f\u00f3rmula o la variable ligada al nodo z es la negaci\u00f3n de la variable ligada al nodo z' .

Esto ser\u00eda la f de que:

$$x \in 3\text{SAT} \iff f(x) \in \text{INDET} \quad (\text{La demo de que esto funciona queda en tu imaginaci\u00f3n})$$

Proposici\u00f3n 10: $\text{CAMHAM} \in \text{NP-completo}$.

Teorema 12: $\text{TSP} \in \text{NP-completo}$.

Proposici\u00f3n 11: $\text{KNAPSACK} \in \text{NP-completo}$

Teorema 13: Si $\text{P} = \text{NP}$ entonces $\text{ExpTime} = \text{NExpTime}$

Demostraci\u00f3n:

$\text{ExpTime} \subseteq \text{NExpTime}$:

Esto ya se cumple sin que $\text{P} = \text{NP}$.

$\text{NExpTime} \subseteq \text{ExpTime}$:

Sea $\mathcal{L} \in \text{NExpTime}$ y $\mathcal{L}(N)$ tal que N es no-det. y corre en $c2^{n^c}$.

Considero $\mathcal{L}_{pad} : \{\langle x, 1^{2^{|x|^c}} \rangle : x \in \mathcal{L}\}$ ($\mathcal{L}$ con padding)

Veamos que $\mathcal{L}_{pad} \in \text{NP}$:

Como la entrada de $\mathcal{L}_{pad}$ es una cadena exponencial con respecto a $|x|$, puedo correr a N con x por $2^{|x|^c}$ pasos porque es poly con respecto a $|\langle x, 1^{2^{|x|^c}} \rangle|$.

Teorema 14: (Jerarquía de Tiempos Determinísticos) Si f, g son construibles en tiempo y cumplen que $f(n) \cdot \log(f(n)) = o(g(n))$ entonces $\text{DTime}(f(n)) \subsetneq \text{DTime}(g(n))$.

Demstración:

A ver, esto va a ser lento y detallado porque me cuesta entenderla.

La idea principal es demostrar por diagonalización que hay lenguajes que estan en $\text{DTime}(g(n))$ pero no en $\text{DTime}(f(n))$.

Declaro D máq. det. tal que:

```

D : ⟨x⟩
  Calcula  $n = |x|$  y  $t = g(n)$        $O(g(n) + n) = O(g(n))$ 
  Simula  $U(\langle x, x \rangle)$  por  $t$  pasos   $O(g(n))$ 
  if( $U(\langle x, x \rangle)$  no terminó en a lo sumo  $t$  pasos):   $O(1)$ 
    ret 0
  else:
    ret  $\neg(U(\langle x, x \rangle))$ 

```

$\mathcal{L} \in \text{DTime}(g(n))$ es inmediato.

Ahora vamos a ver por absurdo la prueba:

Supongo que $\mathcal{L}(D) \in \text{DTime}(f(n))$. Sea $\mathcal{L}(M) = \mathcal{L}(D)$, donde M termina en a lo sumo $O(f(|x|))$ pasos (c.f. $|x|$) acá $|x| \geq n_0$ que es el punto donde siempre la función f se vuelve mayor

Por el **teorema 2** sé que $U(\langle x, x \rangle)$ simula $M_x(x)$ en $c' \cdot c \cdot f(|x|) \cdot \log(c \cdot f(|x|))$ pasos.³

Luego, existen d y $n_1 \geq n_0$ tal que $\forall n \geq n_1$:

$$c' \cdot c \cdot f(|x|) \cdot \log(c \cdot f(|x|)) \leq d \cdot f(n) \cdot \log(f(n)) \leq g(n)$$

Tomo x tal que $|x| \geq n_1$ y $M_x = M$. La existencia de este x es garantizada porque se pueden hacer infinitas máquinas que hagan lo mismo.

Ahora bien, $U(\langle x, x \rangle)$ termina en $g(|x|)$ pasos. Luego, $D(x) = \neg(U(\langle x, x \rangle)) = \neg(M_x(x)) = \neg(M(x))$ **Absurdo! Si habíamos dicho que $\mathcal{L}(D) = \mathcal{L}(M)$**

Ok, tiene sentido, te armás una máquina que corra en $g(|x|)$ pasos, forzas a una máquina a identificar el mismo lenguaje pero en $\log(f(|x|)) \cdot f(|x|)$ pasos y como siempre termina, siempre pasa que da una contradicción con el lenguaje que se suponía que decidía.

¿Qué pasa si le meto D_x ? ¿No me da una contradicción también?

$D(\langle D_x \rangle) = U(\langle D_x \rangle, \langle D_x \rangle)$ por t pasos = $g(|x|)$ pasos ... Opa? Esto no termina, o sea, si que termina, retorna 0 porque no termina el llamado recursivo que se llama a $U(\langle D_x \rangle, \langle D_x \rangle) = D(\langle D_x \rangle)$.

Teorema 15: (Jerarquía de Tiempos no Determinísticos) Si f, g son construibles en tiempo y cumplen que $f(n) \cdot \log(f(n)) = o(g(n))$ entonces $\text{NDTime}(f(n)) \subsetneq \text{NDTime}(g(n))$.

Demstración:

Pendiente.

Teorema 16: (Teorema de Ladner) Si $P \neq NP$ entonces existe $\mathcal{L}$ tal que $\mathcal{L} \in NP$, $\mathcal{L} \notin P$, y $\mathcal{L} \notin NP$ -completo.

Demstración:

Quiero ver que si $P \neq NP \Rightarrow \exists \mathcal{L} : \mathcal{L} \in NP$, $\mathcal{L} \notin P$, y $\mathcal{L} \notin NP$ -completo.

¿Cómo hago esto? Me creo un $\mathcal{L}$ y voy viendo que asumiendo el antecedente $P \neq NP$, llego a obtener que se cumple el consecuente si lo hice bien al $\mathcal{L}$.

Definimos el problema SAT_H relativo a la funcion $H : \mathbb{N} \rightarrow \mathbb{N}$ tal que:

$$\text{SAT}_H = \{\psi 01^{n^{H(n)}} : \psi \in \text{SAT}, n = |\psi|\}$$

Es como un SAT con padding, o sea, no agrega nada a ψ

Ahora, si $H(n) = n \Rightarrow \text{SAT}_h \in P$: Esto pasa porque la entrada sería $\psi 01^{n^n}$, por lo cual, resolver que ψ es SAT en una M det tomaría 2^n , lo cual es polinomial respecto a la entrada (porque $|2^n| \leq |1^{n^n}|$ para toda ψ salvo finitos casos).

³el teorema 2 dice que $U(\langle x, x \rangle)$ corre en $k \cdot z \cdot \log(z)$ donde z es el tiempo de cómputo de x en la máquina M_x

Si $H(n) = \text{constante} \Rightarrow \text{SAT}_H \in \text{ExpTime}$: El tener $\psi 01^{n^c}$ me deja la entrada con $|\psi 01^{n^c}| = n^{c+1}$ y esto no es gran cambio para el SAT original, y sigue siendo NP-completo SAT_H .

Necesito que $H(n)$ tienda a infinito pero que crezca lentamente para llegar a un NP-intermedio.

Se define H tal que:

$$H(x) = \begin{cases} \star \min i < \log(\log(n)) \text{ tal que } \forall x \cdot x \in \{0,1\}^{\leq \log(n)} M_i(x) = \mathcal{X}\text{SAT}_H(x) \text{ en a lo sumo } i \cdot |x|^i \text{ pasos} & \text{si existe tal } i \\ \log(\log(n)) & \text{si no} \end{cases}$$

★ No entendi que hace acá, veamoslo más tranqui:

Buscamos si alguna máquina M_i decide SAT_H sobre todas las entradas chicas ($\leq \log(n)$) en a lo sumo $i|x|^i$ pasos.

El i debe ser el mínimo menor a $\log(\log(n))$

Si no existe, ponemos $H(n) = \log(\log(n))$ para que crezca un poco.

SAT_H y H quedan definidas por recursión mutua (H depende de SAT_H y viceversa)

- $H(n)$ usa $\text{SAT}_H(x)$ para palabras x tal que $|x| \leq \log(n)$
- $\text{SAT}_H(x)$ usa $H(n)$ para palabras x tal que $|x| > n$

donde $x = \psi 01^{n^{H(n)}}$ y $n = |\psi|$